

Laxmi Charitable Trust's
Sheth L.U.J College of Arts & Sir M.V. College of Science and Commerce
Department of Information Technology (Bsc.IT Semester III)
Scala for D.S Practical

Practical - 27,28,29

Roll No. :S039	Name: Divyanshu singh
Class :SY-Bsc.IT	Batch : 00
Date of Assignment :-22-09-2026	Date/Time of Submission : 22-09-2026

PRACTICAL 27

Aim:

Implement one-hot encoding for a categorical column in a dataset.

Code:

```
build.sbt  Demo28.scala x
1  ▶ object Demo28 extends App
2      {
3          val data = List("Red", "Blue", "Green", "Red", "Blue")
4          for (value <- data)
5              {
6                  val r = if (value == "Red") 1 else 0
7                  val b = if (value == "Blue") 1 else 0
8                  val g = if (value == "Green") 1 else 0
9                  println(value + " -> Red:" + r + " Blue:" + b + " Green:" + g)
10             }
11     println("\nDivyanshu Singh S039")
12 }
```

Output:

```
Run Demo28 x
" C:\Program Files\Java\jdk1.8.0_181\bin\java.exe" ...
Red -> Red:1 Blue:0 Green:0
Blue -> Red:0 Blue:1 Green:0
Green -> Red:0 Blue:0 Green:1
Red -> Red:1 Blue:0 Green:0
Blue -> Red:0 Blue:1 Green:0

Divyanshu Singh S039

Process finished with exit code 0
```

PRACTICAL 28

Aim:

Compute the Euclidean distance between two Breeze vectors. Use it for nearest neighbor classification.

Code:

```
ThisBuild / version := "0.1.0-SNAPSHOT"

ThisBuild / scalaVersion := "2.13.12"
libraryDependencies += "org.scalanlp" %% "breeze" % "2.1.0"
lazy val root = (project in file("."))
  .settings(
    name := "Demo28"
  )
```

```

import breeze.linalg._

object Demo28 extends App
{
    val A = DenseVector(2.0, 3.0)
    val B = DenseVector(5.0, 7.0)

    val distance = norm(A - B)

    println("Euclidean Distance: " + distance)

    val C = DenseVector(2.5, 3.5)

    val d1 = norm(C - A)
    val d2 = norm(C - B)

    if (d1 < d2)
        println("Nearest Neighbor: A")
    else
        println("Nearest Neighbor: B")
        println("\nDivyanshu Singh S039")
}

```

Output:

```

"C:\Program Files\Java\jdk1.8.0_181\bin\java.exe" ...
Euclidean Distance: 5.0
Nearest Neighbor: A

Divyanshu Singh S039

Process finished with exit code 0
|

```

PRACTICAL 28

Aim:

Create a simple Spark MLlib pipeline to classify data.
Use logistic regression or decision trees.

Code:

```

build.sbt x Demo29.scala
1 ThisBuild / version := "0.1.0-SNAPSHOT"
2
3 ThisBuild / scalaVersion := "2.13.12"
4
5 libraryDependencies += Seq(
6   "org.apache.spark" %% "spark-sql" % "3.5.9",
7   "org.apache.spark" %% "spark-mllib" % "3.5.9"
8 )
9
10 lazy val root = (project in file("."))
11   .settings(
12     name := "Demo29"
13   )

```

```

build.sbt Demo29.scala x
1 import org.apache.spark.sql.SparkSession
2 import org.apache.spark.ml.feature.VectorAssembler
3 import org.apache.spark.ml.classification.LogisticRegression
4
5 object Demo29 extends App {
6
7   val spark = SparkSession.builder()
8     .appName("Classification")
9     .master("local[*]")
10    .getOrCreate()
11
12   import spark.implicits._
13
14   val data = Seq(
15     (1.0, 0.0),
16     (2.0, 0.0),
17     (3.0, 0.0),
18     (7.0, 1.0),
19     (8.0, 1.0),
20     (9.0, 1.0)
21   ).toDF("marks", "label")
22
23   val a = new VectorAssembler()
24     .setInputCols(Array("marks"))
25     .setOutputCol("features")
26
27   val d = a.transform(data)

```

```

28
29     val model = new LogisticRegression()
30     .setLabelCol("label")
31     .setFeaturesCol("features")
32
33     val result = model.fit(d)
34
35     val prediction = result.transform(d)
36
37     prediction.select("marks", "label", "prediction").show()
38
39     spark.stop()
40 }
41

```

Output:

```

+-----+-----+-----+
|marks|label|prediction|
+-----+-----+-----+
|  1.0|  0.0|        0.0|
|  2.0|  0.0|        0.0|
|  3.0|  0.0|        0.0|
|  7.0|  1.0|        1.0|
|  8.0|  1.0|        1.0|
|  9.0|  1.0|        1.0|
+-----+-----+-----+

```